

Gomoku

Asignatura: **Introducción a la programación**

Curso: **2023/2024**

Estudiante: Jose Nogales Gallardo

Grupo de prácticas: LAB 01

Profesor/a de prácticas: Alberto Gómez Mancha

Convocatoria: Ordinaria (enero)

Índice de contenido

1.Introducción	3
2.Análisis y diseño	4
2.1.Análisis.....	4
2.2.Diagrama modular.....	4
2.3.TAD CASILLA.....	5
2.4.TAD TABLERO.....	6
2.5.TAD JUEGO	7
2.6.Programa principal	8
2.7.Ampliaciones	9
2.7.1.Limitación del número de piezas	9
2.7.2.Ampliar el tamaño del tablero de juego.....	9
2.7.3.Marcar la cadena ganadora.....	10
2.7.4.Ayuda.....	10
3.Planificación y tareas	11
4.Conclusiones y principales problemas.....	12

1. Introducción

En este proyecto final para la asignatura *Introducción a la Programación* de los grados en Ingeniería Informática, el profesorado propone el desarrollo de una versión simplificada del juego de mesa **Gomoku**.

Gomoku es un juego de tablero con unas reglas relativamente sencillas, cuyo principal objetivo es formar una línea de exactamente 5 piezas, ya sea de manera vertical, horizontal, o en diagonal.

Es un juego 1v1, es decir, dos jugadores compiten entre sí por la victoria, y el fin del juego se determinará por la victoria de uno de los jugadores, si el tablero se encuentra completamente lleno de piezas, o si los jugadores han consumido el número de piezas máximo que pueden utilizar antes de conseguir la victoria (en caso de jugar en este modo).

Los jugadores podrán interactuar con el tablero, desplazándose por las intersecciones de este, que actuarán como casillas, y siendo posible colocar una pieza de su color en la casilla deseada, siempre que esta esté vacía y el jugador cuente con piezas suficientes para ello.

2. Análisis y diseño

2.1. Análisis

Dada la temática del presente proyecto de programación, siendo esta la elaboración del juego de mesa **Gomoku**, tendremos que poner el foco sobre la gestión del propio juego.

Al tratarse de un juego de mesa desarrollado sobre un tablero, tendremos que elaborar diferentes TAD para la gestión de este tablero, junto con sus casillas, para poder gestionar la información que se almacena en estas estructuras durante la ejecución del juego.

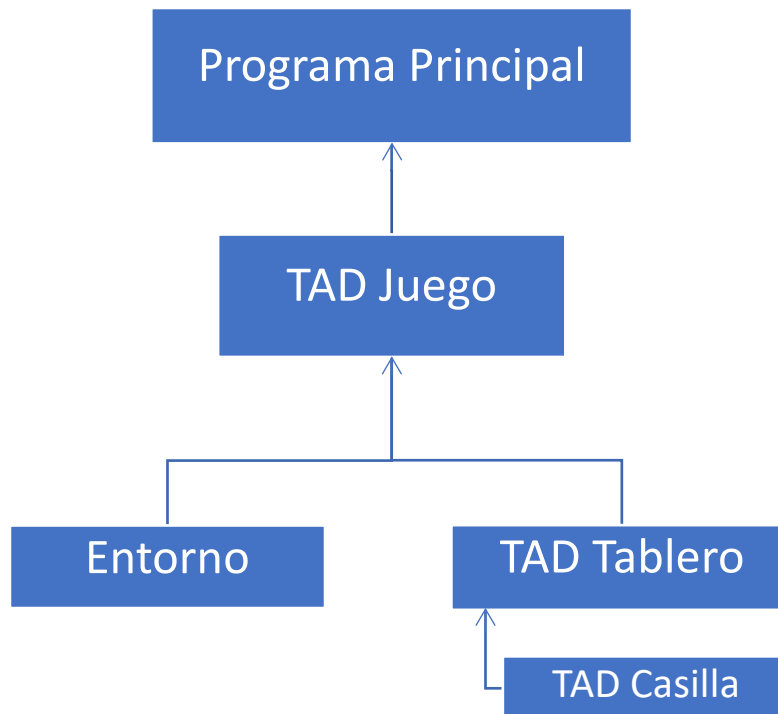
Por lo tanto, en primer lugar, llevamos a cabo la elaboración del **TAD Casilla**, dónde crearemos la estructura de datos correspondiente a cada una de las casillas, guardando su información (si están vacías, o de qué color es la pieza que contiene), y posteriormente, la elaboración del **TAD Tablero**, que hará uso de la estructura previamente creada para las casillas de este (matriz cuadrada de casillas, y el tamaño real utilizado para el tablero), y dentro del cual crearemos los módulos y operaciones que serán necesaria para la gestión de datos del tablero y las casillas que forman este, como comprobar casillas vacías, comprobar el color de la pieza contenida en una casilla, poner una pieza...

Por último, será necesario crear un **TAD Juego**, que utilizará la estructura de datos previamente creada, y almacenará el tablero del propio juego, además de otra información útil, como la longitud de la cadena ganadora, el turno con el que empezará el juego, el tamaño del tablero, y otros datos útiles en ampliaciones posteriores del proyecto, como el número de piezas con el que contará cada jugador. Además de almacenar toda esta información, gestionará el funcionamiento del juego como tal, haciéndose cargo de gestionar el funcionamiento tanto del tablero como de la gestión de los datos almacenados en la estructura de datos, y también actualizando el entorno gráfico a medida que el usuario interactúa con esté y las necesidades de este.

2.2. Diagrama modular

En el siguiente diagrama modular, podemos observar la manera en la que los diferentes TAD y módulos de nuestro programa se relacionan entre sí, y el uso que se hace de ellos, siendo el módulo más importante el TAD Juego, aquel que hace uso del resto de módulos, como *entorno.h*, que hace la gestión del entorno gráfico, actualizando este conforme a las necesidades del juego en ese momento, además del *TAD Tablero (.h y .cpp)*, encargado de gestionar el tablero de juego internamente, para lo que también hace uso del *TAD Casilla (.h y .cpp)*, gestionando esta a su vez cada una de las casillas individuales del juego.

Finalmente, el *main*, o programa principal, realiza la llamada al *TAD Juego*, haciendo así posible la ejecución del programa, y englobando todos y cada uno de los TAD anteriormente mencionados.



2.3. TAD CASILLA

La principal función del TAD Casilla consiste en la creación de cada una de las casillas que utilizaremos en el tablero del juego, además de la gestión de cada una de estas, tanto de su creación, modificación, eliminación y uso.

Dentro del TAD Casilla, encontramos diferentes módulos: crearCasilla, ponerColorCasilla, obtenerColorCasilla y estaVacíaCasilla.

Para comenzar, definimos un struct casilla, dentro del que definimos las siguientes variables:

color: entero (int)

vacía: bool

Estas variables son las encargadas de almacenar la información que contiene cada una de las casillas, como el color que tienen, o la condición de estar borradas o vacías, información necesaria para poder realizar los módulos y gestionar las casillas correctamente.

Pasamos a la descripción de cada uno de los módulos del TAD:

- void crearCasilla(Casilla: c)

efecto: crea una nueva casilla vacía en c

- void ponerColorCasilla(Casilla: c, entero: color)

efecto: pone una pieza del color indicado en la casilla c, siempre que esta esté vacía

- void obtenerColorCasilla(Casilla: c)

efecto: si no está vacía, devuelve el color que contiene la casilla: 0 para negro, 1 para

blanco

- bool esVaciaCasilla(Casilla: c)

efecto: devuelve TRUE si la casilla está vacía, FALSE en caso contrario

Esta TAD además cuenta con su juego de pruebas unitarias, que prueban el correcto funcionamiento de los métodos de este, en el que encontramos pruebas como son: pruebaCrear, pruebaPonerObtenerColor, pruebaEstaVacia, y pruebasCasilla (llamadas a todos los métodos de prueba mencionados anteriormente).

2.4. TAD TABLERO

La principal función del TAD Tablero será la gestión interna del tablero de juego, haciendo uso del previamente creado TAD Casilla, siendo estas casillas las que forman el tablero, y llevando a cabo acciones para la gestión, creación y modificación del tablero en base a las necesidades según las acciones realizadas durante la ejecución del programa.

En este TAD encontraremos diferentes módulos que nos permiten la gestión de este: iniciarTablero, obtenerTamReal, ponerColor, obtenerColor, estaVacia, estaLleno, cuantasVertical, cuantasHorizontal, cuantasDiagonalAbajo y cuantasDiagonalArriba.

En primer lugar, definimos una constante MAX = 19, ya que sabemos que este será el tamaño máximo posible que podrá tener nuestro tablero según la documentación del juego.

A continuación, definimos la estructura de datos Tablero como un struct, que contiene:

tablero: casilla[MAX][MAX] (sabemos que MAX es la constante previamente definida)

tamReal: entero (int)

Procedemos a describir cada uno de los módulos:

- void iniciarTablero(Tablero: t, entero: tam)

efecto: inicia el tablero con el tamaño introducido por parámetro, iniciando cada una de las casillas contenidas en este

- int obtenerTamReal(Tablero: t)

efecto: devuelve el valor del atributo tamReal del tablero (inicializado previamente)

- void ponerColor(Tablero: &t, entero: fila, col, color)

efecto: pone una pieza del color indicado en la casilla que se encuentra en la posición [fila][col] del tablero. Hace uso del método ponerColorCasilla del TAD Casilla

- int obtenerColor(Tablero: t, entero: fila, col)

efecto: devuelve el color que contiene la casilla que está en esa posición del tablero,

0 si es negro, 1 si es blanco. Hace uso del método obtenerColorCasilla del TAD Casilla.

- bool estaVacia(Tablero: t, entero: fila, col)

efecto: devuelve TRUE en caso de que la casilla en la posición *[fila][col]* se encuentre vacía, y FALSE en caso contrario. Hace uso del método estaVaciaCasilla del TAD Casilla.

- bool estaLleno(Tablero: t)

efecto: devuelve TRUE si el tablero está lleno, es decir, todas sus casillas están ocupadas, o FALSE en caso de no cumplirse la condición.

- int cuantasVertical(Tablero: t, entero: fila, col, color)

efecto: cuenta el número de piezas consecutivas que hay del color indicado, a partir de la pieza en la posición indicada, de manera vertical, hacia arriba y hacia abajo.

- int cuantasHorizontal(Tablero: t, entero: fila, col, color)

efecto: cuenta el número de piezas consecutivos que hay del color indicado, a partir de la pieza en la posición indicada, de manera horizontal, hacia la izquierda y hacia la derecha.

- int cuantasDiagonalArriba(Tablero: t, entero: fila, col, color)

efecto: cuenta el número de piezas consecutivos que hay del color indicado, a partir de la pieza en la posición indicada, de manera diagonal, superior hacia la derecha, e inferior hacia la izquierda.

- int cuantasDiagonalAbajo(Tablero: t, entero: fila, col, color)

efecto: cuenta el número de piezas consecutivos que hay del color indicado, a partir de la pieza en la posición indicada, de manera diagonal, inferior hacia la derecha, y superior hacia la izquierda.

Al igual que el anterior TAD, este también cuenta con su juego de pruebas unitarias, probando el correcto funcionamiento de todos los métodos, por medio de las pruebas: pruebaIniciarTablero, pruebaPonerObtenerColorTablero, pruebaEstaVaciaTablero, pruebaEstaLleno, pruebasCuantasVertical, pruebasCuantasHorizontal, pruebasCuantasDiagonalArriba, pruebasCuantasDiagonalAbajo y pruebasTablero (la llamada a todos los módulos anteriores)

2.5. TAD JUEGO

El TAD Juego es el encargado de gestionar el juego en su totalidad, gestionando el tablero de este (haciendo uso del TAD Tablero previamente creado, y dentro del cual también se lleva a cabo la gestión del TAD Casilla), y controlando el turno, la actualización del entorno gráfico del juego (mediante el uso de la librería entorno, facilitada por el profesorado de la asignatura), y las interacciones del usuario con el juego.

En este TAD, encontramos la estructura de datos Juego, compuesta por:

t: Tablero (el tablero utilizado durante el juego)

turno: entero(int), representará el turno correspondiente a la jugada en ese

momento, 0 para el negro, y 1 para el blanco.

cadGanadora: entero(int), representa la longitud que debe tener una cadena de piezas para considerar que se ha ganado el juego.

Está formado por los módulos iniciar, jugar, ganar y terminar.

- void iniciar(Juego: j)

efecto: inicializa la estructura de datos Juego, cargando la información del fichero *gomoku.cnf*, que contiene el tamaño del tablero con el que se va a jugar, el color que comenzará el juego, la longitud de la cadena ganadora, y el número de piezas que están disponible para cada color (se utilizará en una ampliación, pero en el juego base esto no se tiene en cuenta).

Posteriormente, iniciar tanto las estructuras de datos de juego y tablero, como inicializa la interfaz gráfica del juego, y ejecuta una llamada al módulo jugar.

- bool ganar(Juego: j, entero: fila, col)

efecto: cuenta el número de piezas que hacen línea a partir de la indica por parámetro *posición: [fila][col]*, hace uso de los módulos cuantas (vertical, horizontal, diagonalArriba y diagonalAbajo), presentes en TAD Tablero.

Devuelve TRUE si alguno de estos números coincide con la longitud de la cadena ganadora, y FALSE en caso de que ningún número cumpla la condición.

- void terminar(Juego: j, string: msg)

efecto: muestra por pantalla un mensaje de fin de juego, dependiendo cuales hayan sido los motivos de este (ganar el juego, tablero lleno, abandonar el juego, empatar...)

- void jugar(Juego: j)

efecto: gestiona la actualización de la interfaz gráfica y la estructura de datos mediante la pulsación de teclas por parte del usuario, llevando a cabo diferentes comprobaciones: la casilla dónde queremos poner la pieza está vacía, si se ha formado línea completa en alguna de las direcciones (se gana el juego), si el tablero está lleno (se termina el juego), además de controlar la posición por la que se va moviendo el jugador, y terminando el juego si se pulsa ESC.

En ciertas ampliaciones, se añadirá funcionalidad a las teclas F1 y F2, que ahora permanecen sin utilizar.

Este módulo, debido a su complejidad, no cuenta con un juego de pruebas, pero su funcionamiento ha sido comprobado en distintas demos y diferentes escenarios para probar que todo se ejecute como es esperado.

2.6. Programa principal

El programa principal o *main* es la pieza más básica de este proyecto, dónde se llevan a cabo las llamadas a los juegos de pruebas de los TAD casilla y tablero (en la versión final se encuentran comentados tras ejecutarse y comprobar su correcto funcionamiento), además de la llamada al módulo iniciar del TAD Juego (con una estructura del tipo Juego, j, previamente definida), que se encargará de iniciar la estructura de datos y enlazar con los módulos necesarios de ese TAD para llevar a

cabo la correcta ejecución del juego.

2.7. Ampliaciones

2.7.1. Limitación del número de piezas

En esta ampliación, se pide que el número de piezas disponibles de cada color esté limitado, determinado por el último valor leído en el archivo de configuración *gomoku.cnf*, y que en caso de que ambos jugadores gasten todas sus piezas disponibles, el juego termine, indicando que ha habido un empate.

Para ello, hemos tenido que añadir a la estructura de datos del juego un nuevo campo, un entero: *nPiezas*, en el cual se almacena el número de piezas límite para cada color, después de leerse del fichero *gomoku.cnf* a través del método *iniciar* del TAD Juego. Además, en el método *jugar*, tendremos que declarar dos variables de enteros, *nBlancas* y *nNegras*, a las que en un primer momento cuando se ejecuta el programa, se le asignará el valor de *nPiezas* de la estructura de datos del juego.

Se añade también la llamada al método de la librería entorno: *entornoMarcadorNumPiezas(nNegras, nBlancas)* al inicio del bucle *while(!salir)*, previo a la ejecución del método *entornoLeerTecla*.

Esto nos permitirá mostrar el contador en la interfaz gráfica del juego, con el número de piezas disponibles de cada color, y que se irá actualizando.

También tendremos que añadir un control para que se decrementen los valores de estas variables en sus turnos correspondientes, dentro de *case TEnter*, dónde dependiendo del turno en el que se esté jugando, decrementará en 1 las piezas de ese turno y cambiará al otro, además de añadir un caso condicional dónde, si detecta que ambas variables con el número de piezas llegan a 0, el juego finalizará con un mensaje de "Empate".

2.7.2. Ampliar el tamaño del tablero de juego

En el caso de esta ampliación, nos piden que, tras pulsar la tecla F2, que en el juego base no tiene función asociada, el tablero del juego se incremente una fila y una columna, y que esta acción solo se pueda realizar una vez durante la ejecución del juego.

Para ello, debemos añadir al TAD Tablero un nuevo método, *anadirFilaColumna(Tablero &t)*, dónde, se incrementará en 1 la variable *tamReal* que se encuentra dentro de la estructura de datos del tablero pasado por parámetro, y también, inicializamos las casillas correspondientes a la fila y columna que han sido añadidas, para que estas puedan ser utilizadas y recorridas durante la ejecución del juego.

Una vez hecho esto, volvemos al TAD Juego, específicamente al método *jugar*, dónde tendremos que añadir una variable *ampliado*, de tipo booleano, que se inicializa como false y actuará como bandera, y, posteriormente, tendremos que implementar la funcionalidad de la tecla en su apartado correspondiente: *case TF2*.

En este apartado, añadiremos como condición que la variable *ampliado* sea FALSE,

para que así, una vez se ejecuta al pulsar la tecla, se llama a los métodos *entornoAnadirFilaColumna(obtenerTamReal(j.t))*, que añade la fila y columna al tablero mediante el entorno gráfico, y al método *anadirFilaColumna(j.t)* que hemos implementado previamente en TAD Tablero, que se encarga de ampliar internamente el tablero del juego, y para terminar, cambiamos la bandera *ampliado* a TRUE, para que así la condición inicial no se cumpla más durante el resto de la ejecución del programa.

2.7.3. Marcar la cadena ganadora

Para esta ampliación, se pide que, al ganar el juego, la línea ganadora sea marcada en el entorno gráfico con un recuadro rojo a su alrededor antes de finalizar el juego.

Para ello, realizamos modificaciones dentro del módulo *ganar*, que se encuentra en TAD Juego, dónde, en cada uno de los segmentos condicionales, que se ejecutan si la posición que está siendo comprobada forma una línea en alguna de las direcciones, tendremos que añadir bucles similares a los utilizados en estos métodos *cuantas*, que nos permitan recorrer las posiciones, dependiendo de la forma en la que se haya detectado la línea (vertical, horizontal, o diagonal), teniendo en cuenta que todas las fichas sean del mismo color, parando cuando encuentra un espacio vacío u otra ficha de color diferente, y marcando cada una de las piezas que forman esa línea, mediante el módulo *entornoMarcarLinea(fila, col)*, que añadirá este recuadro rojo que nos pedía el enunciado.

2.7.4. Ayuda

En este caso, la ampliación propuesta para realizar nos pide implementar una funcionalidad de ayuda al pulsar la tecla F1, lo cual hará que se marque una posición en el tablero que ayudará al jugador del turno actual a realizar la fila más larga posible en ese momento.

Para ello, realizamos un nuevo módulo dentro de *TAD Juego*, llamado *ayuda*, al cual le pasaremos por parámetro la estructura de datos del juego, y dos parámetros de entrada/salida, *filMejor* y *colMejor*.

En este módulo, tendremos que recorrer la matriz del tablero de juego, buscando cada posición vacía que encontremos, y al encontrarla, llevar a cabo las comprobaciones con nuestros módulos *cuantas*, en todas las direcciones, para ver en que posición vacía podríamos colocar la pieza de este color para formar la línea más larga actualmente posible.

Después, debemos implementar la funcionalidad en el apartado del módulo *jugar* correspondiente con la pulsación de la tecla F1: *case F1*, dónde tendremos que realizar una llamada a este módulo de ayuda, previamente creado, y tras ello, ejecutar el método *entornoMarcarAyuda*, que nos mostrará durante el tiempo establecido por parámetro un indicador azul en la posición que se sugiere que coloquemos esta pieza.

3. Planificación y tareas

Al realizar el proyecto de manera individual, todas las tareas serán realizadas por mi (Jose Nogales Gallardo).

TIEMPO DEDICADO	TAREA
1.5h	Lectura de la documentación inicial
2h	Análisis de los TAD necesarios y diseño de estos.
4h	Diseño e implementación del TAD Casilla (estructura de datos, implementación de operaciones y pruebas)
4h	Definición y diseño del TAD Tablero (estructura de datos y operaciones necesarias)
8h	Implementación del TAD Tablero (juegos de pruebas y operaciones)
10h	Definición e implementación del TAD juego (estructura de datos y operaciones)
6h	Pruebas de integración del proyecto
7h	Ampliaciones del proyecto
5h	Escritura final de la documentación

4. Conclusiones y principales problemas

Algunos de los **principales problemas** a los que me he enfrentado a la hora de llevar a cabo este proyecto son la *falta de un análisis previo* a la implementación de muchas funcionalidades, tratando de implementarlo lo antes posible, sin hacer una reflexión previa sobre cuáles serían las necesidades y los métodos más efectivos para llevarlo a cabo, cosa que puede ayudar a una mejor elaboración del código y una considerable reducción del tiempo a la hora de implementarlo si se hace de manera correcta.

También me parece imprescindible a la hora de hacer proyectos grandes y de este calibre, el saber realizar correctamente una *traza del código* que estamos creando, y usar las *herramientas de depuración* que nos ofrecen los IDE, ya que pueden resultar de gran ayuda a la hora de encontrar problemas tras añadir ciertas funciones, y no encontrar en un primer momento la causa de estos.

Y como **conclusión**, tras la realización de este proyecto de programación, me ha parecido muy interesante poder demostrar todo lo aprendido en la asignatura a través de un reto de tal nivel como la programación de un juego, que ha sido una gran carga de trabajo, pero a su vez ha permitido que saquemos el potencial de todos los aprendizajes obtenidos durante las clases teóricas y prácticas de la asignatura.